Breaking PsycoSupport

Problem statement: classical cryptanalysis of an LLM-backed client–server application

---

Background

PsycoSupport is a small mental-health support application. A patient enters a username, a mood (1–10) and a concern into a client program. The server forwards the concern to a large language model (LLM) and sends supportive advice back to the client.

Because the conversations are sensitive, the client–server traffic is "protected" by a classical encryption wrapper: a variant of the **affine cipher**. The two directions (client→server and server→client) may use different encodings and parameters, and the parameters may depend on the inputs.

The application is available as pre-built Linux executables (`client`, `server`) from `https://github.com/maadilrehman/PsycoSupport`. No source code or protocol documentation is provided.

Goal

Act as a tester-cryptanalyst. Starting only from captured network traffic, determine how the traffic is encrypted, recover the keys, decrypt the conversations, and assess and fix the design.

Tasks

1. **Offline deployment.** Run the client and server on a single Linux machine using only free, local components. The server's original cloud LLM backend is no longer available, so serve a lightweight model (for example `llama3.2`) locally with Ollama, without changing the server's behaviour.
2. **Traffic interception.** Capture the traffic on the loopback interface with `tcpdump` or Wireshark, filtered to the application's port. Generate at least five sessions with varied moods and concerns, and extract the raw ciphertexts in both directions (e.g. with *Follow TCP Stream*).
3. **Frequency analysis (Al-Kindi method).** For each ciphertext, and separately for each direction, remove irrelevant characters, compute letter counts and relative frequencies, present them as tables and charts, and compare them with standard English letter frequencies to propose plaintext–ciphertext mappings.
4. **Key recovery.** From the proposed mappings, form linear congruences $E(x) = (ax+b) \bmod 26$, solve them for (a, b), check $\gcd(a, 26) = 1$, compute $a^{-1} \bmod 26$, and test candidate keys against the ciphertext. Give the full mathematical working and justify the final key for each direction and input condition.
5. **Decryption.** Decrypt every captured session into readable plaintext, showing the key used and the decryption steps, with examples from both directions.
6. **Generalisation.** Describe how the key depends on the inputs (username, mood, direction) and build a general method or script that can decrypt any message from the application.

7. **Security analysis and fixes.** Explain why predictable LLM output (fixed openings, boilerplate, formatting) makes classical substitution ciphers weak against frequency and known-plaintext attacks. Propose, and ideally demonstrate, standard local protections for the inter-process traffic, such as TLS 1.3 or UNIX domain sockets.

Expected outputs

- A list of the captured ciphertexts (`ciphertexts.csv`) and capture evidence (screenshots, `.pcap`).
- Analysis scripts for extraction, frequency analysis, charts and decryption.
- A report with the tables, charts, key derivations, decrypted texts, security analysis and a reflection on ethics.

Rules of engagement

- Analyse only traffic from your own instance of the application, on machines and networks you control.
- Use synthetic messages only, and never submit or commit real personal data or API keys.
- Packet capture and interception tools can be misused. Intercepting traffic of real applications or users without authorisation is unethical and illegal.